CS-DCC
Climate-Smart Data Collaboration Centre

TERM_HARVESTER.PY

Term Harvester

A python script to synchronize agency- or research-project-managed vocabularies with known-good online resources.

Damion Dooley
damion_dooley@sfu.ca
GitHub repo: github.com/ClimateSmartAgCollab/term-harvester
May 13, 2025

CS-DCC
Climate-Smart Data Collaboration Centre

A configuration-driven assembler of controlled vocabulary resources

Data specifications need to draw upon established or de-facto standards for measurement and metadata variable content, **including picklist choices**.

Online sources of controlled vocabulary span **ontologies**, SKOS vocabularies like **AGROVOC**, terminology portals like **BioPortal** or the EMBL-EBI **Ontology Lookup Service**, and even web pages like **STATSCAN** or PDF documents from **USDA**.

Organizing and keeping all of this up-to-date is a challenge. `term_harvester.py` and its `sources/` fetch-and-parse library retrieve known-good vocabulary by configuration, for a single project or an agency.

EXTENDING

A *new kind* of vocabulary resource — beyond the ten content types in this deck — requires a new `source_*.py` parser. That's a programming task, typically done in collaboration with Claude.

SHAPE OF THE PROBLEM

Config file	`harvester_config.yaml`
Hierarchies	`parent / is_a chains`
Selectivity	`minus + include`
Status	`skip obsolete / deprecated`
Re-fetch	`version + download_date`
Common form	`LinkML enums`
Assembly	`→ schema.yaml`

CS-DCC
Climate-Smart Data Collaboration Centre

Four top-level blocks; the rest is `sources`

BLOCK · DICT

apis:

Endpoint routing for `-l`. Each entry has a `type` (rest / sparql), `uri`, optional `apikey`, and an `ontologies` list of CURIE prefixes it handles.

```yaml
apis:
  ols:
    type:
      rest:
        uri: .../ols4/api/...
    ontologies: [ENVO, GO]
  agrovoc:
    type:
      sparql:
        uri: agrovoc.fao.org/sparql/
```

BLOCK · LIST

sssom:

SSSOM mapping files applied to `schema.yaml` by `-s`. Each entry is a local path or HTTPS URL to a TSV.

```yaml
sssom:
  - ./sources/sssom.tsv
  - https://w3id.org/.../map.tsv
```

BLOCK · LIST

locales:

Language codes that source parsers should capture when bilingual content is available (e.g. STATSCAN, AgriFoodCA EN/FR).

```yaml
locales: [en, fr]
```

BLOCK · DICT OF DICTS

sources:

The main payload. One key per vocabulary source, each value is a dict of metadata, fetch settings, and optional filters — see attributes below.

```yaml
sources:
  envo:
    content_type: OWL
    file_format: owl
    reachable_from:
      source_ontology: .../envo.owl
    version: '2025-10-20'
    download_date: '2026-04-16'
```

Per-source attributes `sources.{key}.…`

REQUIRED

`content_type` `file_format` `reachable_from.source_ontology`

Drive fetching and parsing. Auto-filled by `-a` based on URL or content detection.

PROVENANCE & METADATA

`name` `title` `version` `download_date` `description` `license` `see_also` `prefixes`

Filled during `-a` / `-c`; propagated into `schema.yaml` on `-b`.

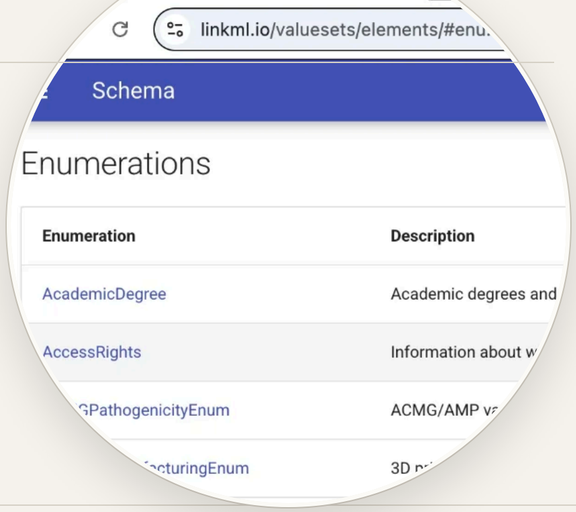
OPTIONAL FILTERS

`minus.concepts` `minus.permissible_values` `minus.status` `include.concepts` `include.permissible_values` `concise: true`

`include` without `minus` = whitelist. `concise` drops `status: obsolete` PVs.

CS-DCC
Climate-Smart Data Collaboration Centre

LinkML



ONTOLOGY BACK-END

`.yaml`

DETECTION

YAML dict containing `enums` or `id` at the top level.

ABOUT

The canonical native format. The `linkml/valuesets` project is a large curated bundle of merged value sets ready to import.

PARSED YAML FRAGMENT

sources/valuesets_merged_hierarchy.yaml

```yaml
enums:
  PeerReviewStatus:
    title: Peer Review Status
    description: Status of peer review process
    status: DRAFT
    permissible_values:
      SUBMITTED:
        description: Submitted for review
      UNDER_REVIEW:
        title: ongoing
        meaning: 'SIO:000035'
      ACCEPTED:
        description: Accepted for publication
      PUBLISHED:
        title: publication
```

SINGLE ENUM / BRANCH

One enum (whitelist)

Without `minus:`, `include:` acts as a whitelist.

```yaml
# harvester_config.yaml
valuesets_merged_hierarchy:
  content_type: LinkML
  include:
```

WHOLE RESOURCE

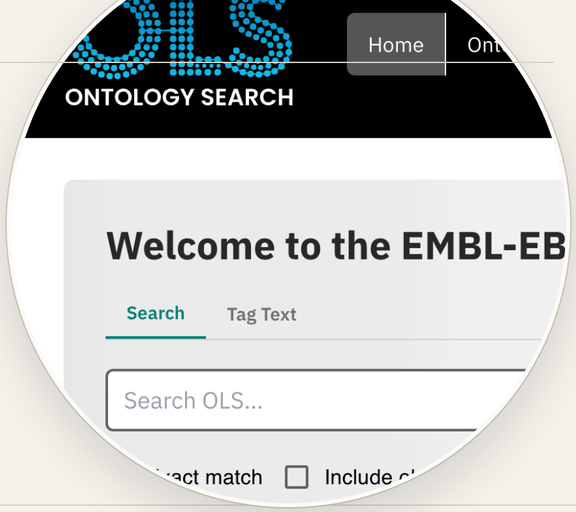
Whole bundle

One URL pulls every merged enum and its prefixes.

```
python term_harvester.py -a https://raw.githubusercontent.com/linkml/valuesets/refs/heads/main/src/valuesets/merged/merged_hierarchy.yaml
```

CS-DCC
Climate-Smart Data Collaboration Centre

OntologyAPI



ONTOLOGY BACK-END

REST · SPARQL

DETECTION

Bare CURIE, OBO IRI, AGROVOC IRI, or SNOMED IRI — routed by prefix to OLS4 / BioPortal / AGROVOC.

ABOUT

No file is downloaded. The `apis` block maps each prefix to a REST or SPARQL endpoint; `-l` expands the chosen hierarchy.

PARSED YAML FRAGMENT

sources/AGROVOC_c_2808.yaml

```yaml
enums:
  AGROVOC_c_2808:
    title: farmland
    reachable_from:
      api:
        agrovoc:
          type: sparql
      source_nodes: ['agrovoc:c_2808']
    permissible_values:
      'agrovoc:c_18948':
        title: arable land
        is_a: 'agrovoc:c_2808'
      'agrovoc:c_34007':
        title: fallow
        is_a: 'agrovoc:c_2808'
```

SINGLE ENUM / BRANCH

One concept + its subtree

Add the term, then expand only its descendants.

```
# OBO via OLS4
python term_harvester.py -a ENVO:00010483
python term_harvester.py -l ENVO_00010483
```

WHOLE RESOURCE — N/A

WHY

API-driven by design — hierarchies fetch on demand.

PATTERN

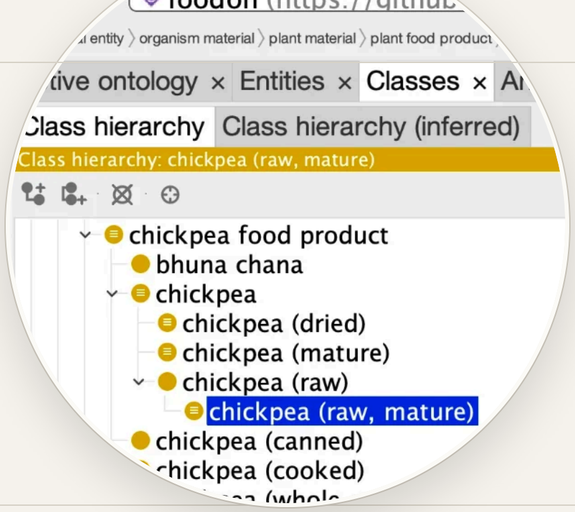
Add entry concepts; let `-l` walk only their subtrees.

MISSION FIT

Manage only **domain-relevant** vocabulary; a full ontology dump defeats the goal.

CS-DCC
Climate-Smart Data Collaboration Centre

OWL



ONTOLOGY BACK-END

`.owl .ofn .rdf .ttl .n3`

DETECTION

URL extension is an RDF/OWL serialisation, or the file body contains RDF/OWL markers.

ABOUT

Parsed via `owlready2`. `minus` / `include` lists filter classes by English `rdfs:label`.

PARSED YAML FRAGMENT

sources/envo.yaml

```yaml
enums:
  envo:
    permissible_values:
      ENVO_00000077:
        title: agricultural ecosystem
        is_a: ENVO_01001828
        meaning: 'ENVO:00000077'
      ENVO_00000078:
        title: farm
        description: An area of land used for the
          cultivation of crops or grazing of livestock.
        is_a: ENVO_00000077
        meaning: 'ENVO:00000078'
      ENVO_00000115:
        title: orchard
```

SINGLE ENUM / BRANCH

One branch — water body

Exclude a top-level branch, restore a sub-subtree.

```
# harvester_config.yaml
Envo:
  content_type: OWL
  reachable_from:
```

WHOLE RESOURCE

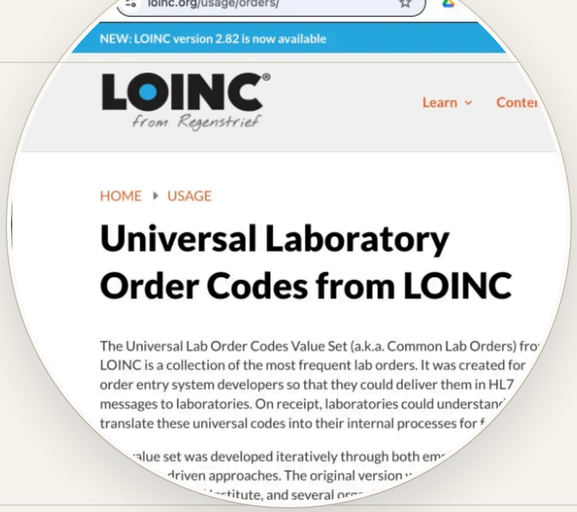
Whole ontology

Possible, but ENVO and friends are large — usually you filter.

```
python term_harvester.py -a https://purl.obolibrary.org/obo/envo.owl
```

CS-DCC
Climate-Smart Data Collaboration Centre

LOINC



HL7 TERMINOLOGY

`.html`

DETECTION

`terminology.hl7.org` URL ending in `.html` that is a listing page (not a single detail).

ABOUT

HL7 publishes `valuesets-*.html` listings; `-c` walks the table and fetches each value-set's JSON.

PARSED YAML FRAGMENT

sources/LOINCValuesetsCda.yaml

```yaml
enums:
  LOINC_ActClass:
    description: A code specifying the major type
      of Act that this Act-instance represents.
    status: ACTIVE
    permissible_values:
      ACT:
        title: act
        description: A record of something being
          done, has been done, or can be done.
      DOC:
        title: document
      DOCCLIN:
        title: clinical document
      EHR:
```

SINGLE ENUM / BRANCH

One value-set from the listing

Add the JSON URL directly — auto-detects as LOINCValueSet.

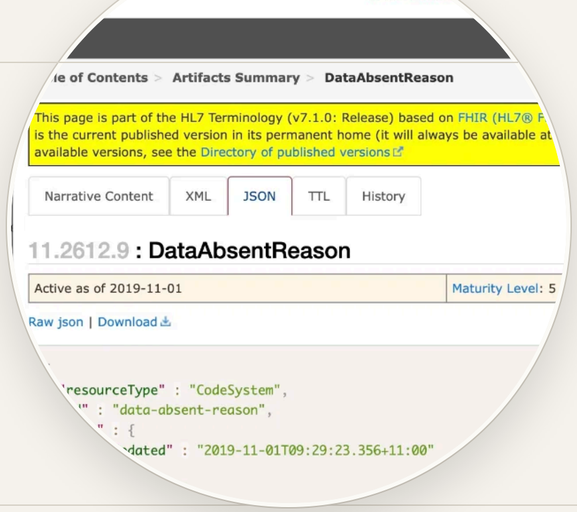
```
python term_harvester.py -a https://terminology.hl7.org/7.1.0/en/ValueSet-pronouns.json
```

WHOLE RESOURCE

Every value-set on the listing

Two-step: register the listing page, then process it.

```
python term_harvester.py -a https://terminology.hl7.org/en/valuesets.html
python term_harvester.py -c LOINCValuesets
```



CS-DCC
Climate-Smart Data Collaboration Centre

LOINCCodeSystem · LOINCValueSet via HL7

HL7 TERMINOLOGY

`.json (FHIR)`

DETECTION

`.json` file whose `resourceType` is **CodeSystem** or **ValueSet**.

ABOUT

One URL = one enum bundle. Auto-renamed from the FHIR `name` field; prefix `LOINC:` applied to all CURIEs.

PARSED YAML FRAGMENT

sources/LOINC_DataAbsentReason.yaml

```yaml
enums:
  LOINC_DataAbsentReason:
    description: Used to specify why the normally
      expected content of the data element is missing.
    status: active
    permissible_values:
      unknown:
        title: Unknown
        description: The value is expected to exist
          but is not known.
      asked-unknown:
        title: Asked But Unknown
        is_a: unknown
      not-applicable:
        title: Not Applicable
```

SINGLE ENUM / BRANCH

One enum / one CodeSystem

The single resource is already the unit of import.

```
# Data Absent Reason — CodeSystem
python term_harvester.py -a https://terminology.hl7.org/7.1.0/en/CodeSystem-data-absent-reason.json
```

NO BULK OPTION

SELF-CONTAINED

Each **CodeSystem** / **ValueSet** JSON is one complete bundle.

FOR MANY AT ONCE

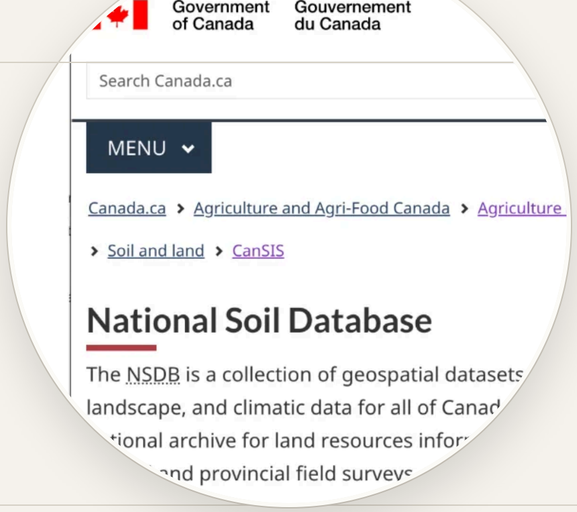
Use the `LOINC` listing-page flow on the previous slide.

PATTERN

Add each JSON URL directly — auto-named from the FHIR `name`.

CS-DCC
Climate-Smart Data Collaboration Centre

NSDB family



GOVT. OF CANADA

`.html`

DETECTION

`sis.agr.gc.ca/cansis/nsdb/` URLs — sub-paths route to **NSDB**, **NSDBSNT**, **NSDBSLT**, and **NSDBSLC**.

ABOUT

AAFC's National Soil DataBase, scraped from HTML attribute pages. Each table contributes one enum group; bilingual EN/FR captured.

PARSED YAML FRAGMENT

sources/NSDBSNT_WATERTBL.yaml

```yaml
enums:
  NSDB_WATERTBL:
    title: Water Table Characteristics
    description: Indicates the presence of a water
      table at a depth of 100cm.
    permissible_values:
      YB:
        title: Always
      YG:
        title: Growing season
      YN:
        title: Non growing season
      'NO':
        title: Never
      '-':
```

SINGLE ENUM / BRANCH

Just water-table characteristics

Drill straight to a single attribute page within SNT.

```
python term_harvester.py -a https://sis.agr.gc.ca/cansis/nsdb/soil/v2/snt/watertbl.html
```

WHOLE RESOURCE

Combined Soil Name + Layer tables

Top-level NSDB index registers both tables as one source.

```
python term_harvester.py -a https://sis.agr.gc.ca/cansis/nsdb/soil/v2/index.html
```

CS-DCC
Climate-Smart Data Collaboration Centre

STATSCAN

GOVT. OF CANADA

`.html`

DETECTION

`statcan.gc.ca` URLs containing `p3VD.pl` with `Function=getVD`. Each URL is one classification variable.

ABOUT

The parser follows each classification code into its *Display structure* and *Display definitions* pages to assemble the full hierarchy.

PARSED YAML FRAGMENT

sources/STATSCAN_1326727.yaml

```yaml
enums:
  STATSCAN_1326727:
    description: ''
    permissible_values:
      '1':
        title: Man
        description: Persons whose reported gender
          is male. Includes cisgender and transgender
          men.
      '2':
        title: Woman
        description: Persons whose reported gender
          is female. Includes cisgender and transgender
          women.
      '3':
```

SINGLE ENUM / BRANCH

One classification variable

Browse `statcan.gc.ca/en/concepts/search`, click a variable, paste its `p3VD.pl` URL.

```
python term_harvester.py -a "https://www23.statcan.gc.ca/imdb/p3VD.pl?Function=getVD&TVD=1368814"
python term_harvester.py -c STATSCAN_1368814
```

VARIABLE-BY-VARIABLE

NO MASTER BUNDLE

STATSCAN exposes one classification per URL.

PATTERN

Add each variable by its `p3VD.pl` URL.

MISSION FIT

Curate only the **domain-relevant** variables a project uses.

CS-DCC
Climate-Smart Data Collaboration Centre

NAPCSCanada

GOVT. OF CANADA

.CSV

DETECTION

CSV headers match the NAPCS schema; the year is extracted from the URL to form the source key.

ABOUT

North American Product Classification System — a hierarchical product taxonomy. `concise: true` drops redundant nodes whose title matches their parent.

PARSED YAML FRAGMENT

sources/NAPCSCanadaAgriculturalGoods2022.yaml

```yaml
enums:
  NAPCSCanadaAgriculturalGoods2022:
    permissible_values:
      '111':
        title: Live animals
      '11111':
        title: Cattle and calves
        is_a: '111'
      '1111111':
        title: Cattle
        description: Live beef and dairy cattle,
          1 year and over.
        is_a: '111111'
      '11111111':
        title: Cattle for slaughter
```

SINGLE ENUM / BRANCH

A subset by code

Whitelist specific product codes via `include.permissible_values`.

```
# harvester_config.yaml — cattle-only subset
NAPCSCanada2022:
  content_type: NAPCSCanada
  concise: true
```

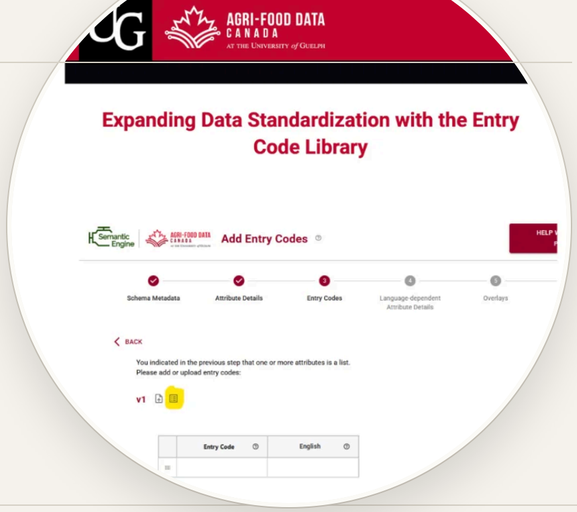
WHOLE RESOURCE

Full 2022 product hierarchy

Auto-detected from CSV headers; year inferred from URL.

```
python term_harvester.py -a "https://www.statcan.gc.ca/en/media/5274"
python term_harvester.py -c NAPCSCanada2022
```

AgriFoodCA



COMMUNITY PICKLISTS

`.csv · GitHub dir`

DETECTION

`github.com/agrifooddatacanada/picklists_for_schemas` directory URL, or CSV with `,title,description,keywords,source` first row.

ABOUT

Community-curated bilingual EN/FR picklists. Directory imports use the GitHub API to fetch every CSV in one pass.

PARSED YAML FRAGMENT

`sources/AgriFoodCA_SoilAerationStatus.yaml`

```yaml
enums:
  AgriFoodCA_SoilAerationStatus:
    permissible_values:
      AERW:
        title: Well-aerated
        description: Good oxygen availability
      AERM:
        title: Moderately aerated
      AERP:
        title: Poorly aerated (anaerobic)
extensions:
  locales:
    value:
      fr:
        enums:
```

SINGLE ENUM / BRANCH

One picklist

Source key derived from filename: `soil_drainage.csv → AgriFoodCA_SoilDrainage`.

```
python term_harvester.py -a https://raw.githubusercontent.com/agrifooddatacanada/picklists_for_schemas/main/picklists/soil_drainage.csv
```

WHOLE RESOURCE

Every picklist in the directory

One `-a` on the directory URL imports them all.

```
python term_harvester.py -a https://github.com/agrifooddatacanada/picklists_for_schemas/tree/main/picklists
```

CS-DCC
Climate-Smart Data Collaboration Centre

NASIS · NRCS Soil Field Book

USDA NRCS

.pdf

DETECTION

`nrcs.usda.gov` URLs containing *NASIS* or the Field Book, ending in `.pdf`. Parsed via `pypdf`.

ABOUT

USDA NRCS publishes domain tables as one massive PDF. NASIS 7.4.x yields ~453 enums across 906 pages; `concise: true` drops ~1,950 obsolete-marked PVs.

PARSED YAML FRAGMENT

sources/NRCSSoilFieldBook.yaml

```yaml
enums:
  NRCSSoilFieldBook_SalinityClass:
    title: Salinity Class
    description: Soil salinity classes based on
      electrical conductivity from a saturation
      paste extract.
    see_also: .../Field-Book-Ver4.pdf#page=141
    permissible_values:
      '0':
        title: nonsaline
        description: ECe < 2 dS/m
      '1':
        title: very slightly saline
        description: ECe 2 to < 4 dS/m
      '4':
```

SINGLE ENUM / BRANCH

One domain by name

After processing, whitelist specific domains.

```yaml
# harvester_config.yaml
NASIS:
  content_type: NASIS
  concise: true
```

WHOLE RESOURCE

Every NASIS domain

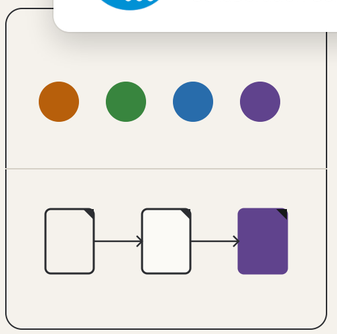
Single `-a` registers the PDF, saves it, and runs the parser.

```
python term_harvester.py -a "https://www.nrcs.usda.gov/sites/default/files/2025-07/NASIS%207.4.3%20Domains.pdf"
```

CS-DCC
Climate-Smart Data Collaboration Centre

Parameter flowchart for `term_harvester.py`

A left-to-right timeline of every CLI option and the files each one reads and writes — from the initial `-a` that adds a source, through maintenance steps `-f -c -l -r -d`, to the final `-b` build and optional `-s` SSSOM mapping pass.



SCOPE
9 CLI options · 5 lifecycle groups

AUDIENCE
Schema / bioinformatics collaborators

ENDS AT
`schema.yaml` (LinkML)

CS-DCC
Climate-Smart Data Collaboration Centre

Add → Fetch → Maintain → Build → Map

CLI OPTIONS

STAGE 01 — Register

-a `--add` — ADD

Add sources by URL; auto-detects content type and appends a stub entry to `harvester_config.yaml`.

`URL …` `nargs=+`

STAGE 02 — Fetch & process

-f `--fetch` — FETCH

Download source files. `-f all` fetches every source; bare keys fetch a subset; alone, defers to `-c`'s keys.

`all` `KEY …` `(none)` `nargs=*`

-c `--config` — PROCESS

Parse downloads into `sources/{key}.yaml`; refresh prefix dicts in harvester_config.yaml.

`KEY …` `(all)` `nargs=*`

STAGE 03 — Maintain

-l `--lookup` — EXPAND

Expand `reachable_from.source_nodes` via the `apis` block (OLS4 / BioPortal REST · AGROVOC SPARQL). Accepts source keys or enum names.

`KEY …` `ENUM …` `(none)` `nargs=*`

-r `--report` — REPORT

Emit an enum report across every source YAML. Pair with `-t` for TSV.

`store_true`

-d `--delete` — DELETE

Remove source keys from harvester_config.yaml; strip their enums from schema.yaml.

`KEY …` `nargs=+`

STAGE 04 — Build schema

-b `--build` — BUILD

Create or update `schema.yaml` — upserts enums, syncs prefixes, flags missing enums for manual review.

`KEY` `(all sources)` `nargs=?`

-t `--tabformat` — MODIFIER

Switches `-r` / `-f` report output to tab-delimited TSV.

`store_true`

STAGE 05 — Map (SSSOM)

-s `--sssom` — MAP

Apply SSSOM mappings to permissible_values in `schema.yaml`. Runs *after* `-b`. Predicate args narrow which mapping types are written.

`exactMatch` `closeMatch` `broadMatch` `narrowMatch` `relatedMatch` `nargs=*`

FILES & DATA

CONFIG · APPENDED
harvester_config.yaml
New source entry: URL, content_type, file_format, prefixes.

REMOTE · DOWNLOADED
sources/{key}.{yaml|json|html|owl|csv|pdf}
Raw file from `reachable_from.source_ontology` (OntologyAPI sources fetch via API, no file).

LINKML INTERMEDIARY
sources/{key}.yaml
Normalized per-source enums & prefixes.

CONFIG · SPINE
harvester_config.yaml
Read for apis block; rewritten on `-c` / `-d`.

API · READ
OLS4 · BioPortal · AGROVOC
REST and SPARQL endpoints routed via `apis` block; `-l` expands source_nodes.

SCHEMA · UPSERTED
schema.yaml
All source enums merged; prefixes synced; stale enums reported.

MAPPING INPUT · OPTIONAL
sources/sssom.tsv
Listed under top-level `sssom:` in harvester_config.yaml.

FINAL OUTPUT
schema.yaml
permissible_values gain exact/close/broad/narrow/related_mappings.

-a add · -f -c fetch / process · -l -r -d maintain · -b build · -s map (SSSOM) · -t modifier · File / data

CS-DCC
Climate-Smart Data Collaboration Centre

What each flag accepts

-a --add

nargs=+

One or more URLs; content type auto-detected from suffix or body.

- `URL` .yaml · .json · .html · .owl · .csv · .ofn · .rdf · .ttl

-f --fetch

nargs=*

Downloads registered sources into `sources/`.

- `all` every source in harvester_config.yaml
- `KEY …` a subset of source keys
- `(none)` defers to the keys passed with -c

-c --config

nargs=*

Process downloads and refresh prefix dicts.

- `KEY …` process only the named sources
- `(none)` process every source

-l --lookup

nargs=*

Expand `reachable_from.source_nodes` via the `apis` block — routes by CURIE prefix to OLS4 / BioPortal REST or AGROVOC SPARQL.

- `SOURCE_KEY` all enums imported_from that source
- `ENUM_NAME` a specific enum in schema.yaml
- `(none)` every enum with source_nodes

-r --report

store_true

Enum report across every source YAML.

- `flag` no arguments; pair with -t for TSV

-d --delete

nargs=+

Removes entries from config and enums from schema.

- `KEY …` required; one or more source keys

-b --build

nargs=?

Create or update schema.yaml.

- `SOURCE_KEY` rebuild only that source's enums
- `(none)` rebuild from every source

-s --sssom

nargs=*

Apply mappings from the top-level `sssom` file list.

- `skos:exactMatch` → exact_mappings
- `skos:closeMatch` → close_mappings
- `skos:broadMatch` → broad_mappings
- `skos:narrowMatch` → narrow_mappings
- `skos:relatedMatch` → related_mappings

-t --tabformat

store_true

Modifier. Switches report/fetch output from padded columns to TSV.

- `flag` pairs with -r or -f

CS-DCC
Climate-Smart Data Collaboration Centre

Four end-to-end recipes

Register a brand-new source

STAGE 01

Adds a URL, auto-detects type, and writes a stub entry into harvester_config.yaml.

```
# Add NAPCS Canada 2022 from its CSV
python term_harvester.py -a "https://www.statcan.gc.ca/en/media/5274"
```

Full refresh of one source

STAGES 02 → 04

Fetch, process, then rebuild schema — scoped to a single key.

```
python term_harvester.py -f NAPCSCanada2022 -c NAPCSCanada2022 -b NAPCSCanada2022
```

Rebuild everything & map

STAGES 02 → 05

Fetch all sources, reprocess, rebuild schema, then apply every SSSOM predicate.

```
python term_harvester.py -f all -c -b -s
```

Add & expand an ontology term

STAGES 01 + 03

OntologyAPI sources accept a bare CURIE or OBO IRI; `-l` walks `reachable_from.source_nodes` via the routed API (OLS4, BioPortal, or AGROVOC SPARQL).

```
python term_harvester.py -a ENVO:00010483      # OBO via OLS4
python term_harvester.py -a http://snomed.info/id/56265001
python term_harvester.py -l ENVO_00010483      # expand hierarchy
```